DPI343
Computer Organization and Assembly Language

Lecture 3

Lectures - prof. em. U. Sukovskis

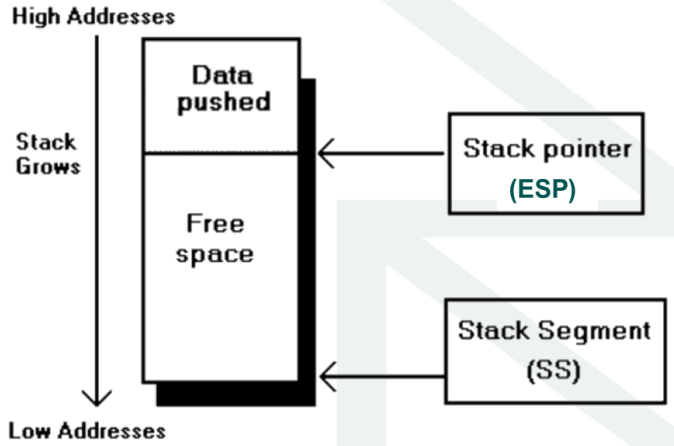
Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign studdents - assoc. prof. G. Alksnis

Autumn 2025

RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

General Purpose Instructions. PUSH and POP



The value of the ESP register is address of the top of the stack (most recently pushed, first to be popped item). Older stack items reside at higher addresses.

PUSH operand

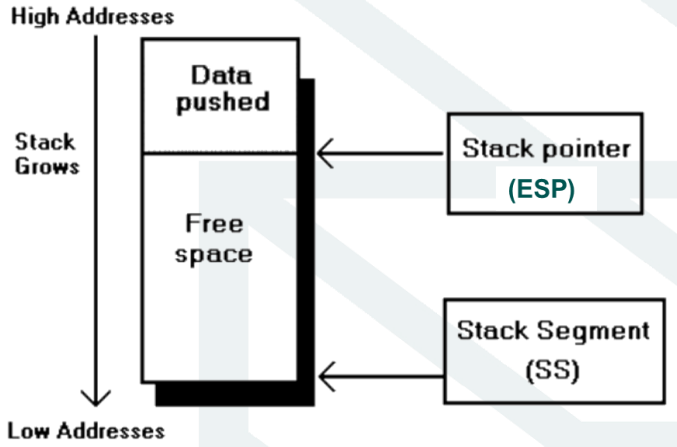
Pushes value of the operand onto the stack. Stack pointer register is decreased by the length of the operand (2 for 16-bit item or 4 for 32 bit item). The operand can be register, memory location, or an immediate value.

POP operand

Pops value from the top of the stack into operand. Stack pointer register is increased by the length of the operand (2 for 16-bit item or 4 for 32 bit item).

The operand can be register or memory location.

General Purpose Instructions. PUSH and POP



- Stack memory is often used:
  - to temporarily store register values and then restore them,
  - to pass parameters to procedures,
  - to temporarily store intermediate results of calculations.

- A good principle is to always use only the same size stack items in push and pop instructions in one program. Try not to mix word and double word items in the stack unless there is a special need
- The stack must be in balance within one procedure - as many bytes are inserted into the stack with push, the same number of bytes must be taken with pop instructions.
- This means that the value of the stack pointer ESP (i.e. the top of the stack) at the end of the procedure (before returning to the calling procedure) must be exactly the same as when entering the procedure.
- We'll learn more about stack and parameter passing later (Lecture6).

Text processing example

- TASK: Find a position of the first symbol * in the text string given.
  Put the result on screen as an integer value

- Some text: "Symbol * is asterisk"
- Result should be 8

- Lets compare each symbol of the text with symbol * until we reach the * or reach the end, meaning symbol not found
- How to define a text string in Assembly program?

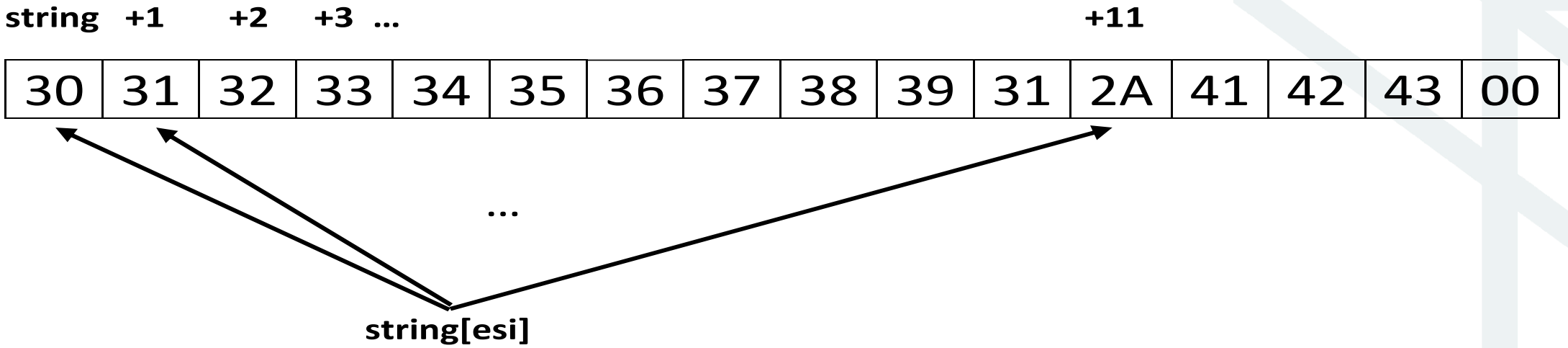
```
string    byte   '01234567891*ABC', 0
```

- Symbol * is at position 12 and the trailing byte of the text contains binary zero

Find the star

```
string     byte    '01234567891*ABC', 0
```

Internal code of this text, starting from the address **`string`**:



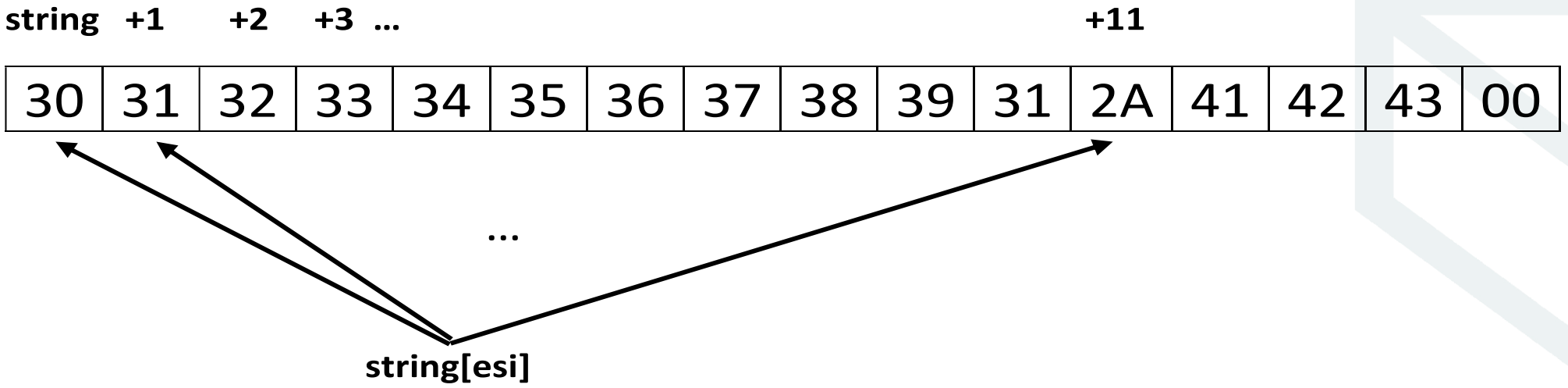
Name **`string`** is the address of the first byte (which in our case contains internal code of the symbol 0).

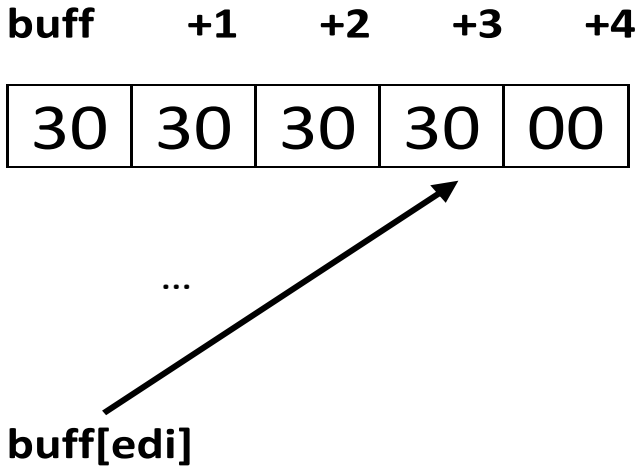
Address **`string[esi]`** is calculated by CPU as address **`string`** plus current value of the register ESI

Find the star (1)

```
;Find a position of the symbol * in the text string given and output it on the screen

.data
string    byte '01234567891*ABC', 0
buff      byte '0000', 0
msg       byte 'Star not found!', 0

.code
start proc
          mov  ah, '*'        ; symbol to find
          mov  esi, 0         ; index of the symbol to check in the string

check:    cmp  string[esi], 0     ; compare the current symbol with binary zero
          je   notfound
          cmp  ah, string[esi]    ; compare symbol in ah with a current byte
          je   found
          inc  esi           ;index = index+1
          jmp  check

found:    mov  eax, esi       ; copy index to eax
          inc  eax            ; calculate number of the position
```

Find the star





How to covert an integer value to text string suitable for output

Decimal value of the register EAX is 12

Internal code of the register EAX

To do:

1) calculate each digit of the decimal value
   - divide the value by the base of the decimal system (i.e. 10) – remainder is the next digit, then divide again the quotient and continue until result is 0

     12 / 10 = 1 and remainder is **2**

     1 / 10 = 0 and remainder is **1**

2) convert each digit to ASCII code representing the decimal digit and store the code of the symbol in the output buffer

   numeric value 2 -> 32h = ASCII code of the symbol ‘2’

   numeric value 1 -> 31h = ASCII code of the symbol ‘1’

	+0	+1	+2	+3	+4
buff	30	30	31	32	00

Find the star (2)

```
; Convert the binary value in register AX to a decimal number representation in a string of symbols

     mov  edi, 3         ; index of the last symbol in the output buffer
     mov  bl, 10         ; divisor - base of the decimal system

d:   div  bl             ; ax/bl= ah-remainder,al-quotient
     add  ah, 30h        ; convert to ASCII

     mov  buff[edi], ah ; store symbol of the digit into output buffer

     cmp  al, 0          ; quotient = 0 ?
     je   done           ; yes, stop looping

     dec  edi            ; next index in the output buffer

     mov  ah, 0          ; clean high byte in ax to have next value to divide by 10
     jmp  d

done:
```

What is the maximum decimal number that can be converted this way?

Find the star (3)

```
done:
        push offset buff        ; parameter for function puts - address of buff
        jmp  show
notfound:
        push offset msg         ; parameter for function puts - address of msg

show:   call puts               ; call C function puts for console display
        add  esp, 4

        push 0
        call ExitProcess@4

start   endp
        end  start
```

How to avoid displaying leading zeros?

Thank You!